

Simulação e Teste de Software

Laboratório 6b: Partição de Equivalência, Valor-Limite e Tabela de
Decisão

Prof. Luciano Rossi

Departamento de Ciência da Computação
Centro Universitário FEI

2º Semestre de 2026

🎯 Objetivo

Segunda rodada de prática das técnicas caixa-preta da aula 6, sobre um único cenário: a cobrança do estacionamento de um shopping.

- Classes de equivalência e valores-limite em uma regra com arredondamento
- Diagnóstico de defeitos de fronteira em uma implementação pronta
- Tabela de decisão com quatro condições e redução por don't care
- Escolha dos valores concretos de cada regra reduzida

Preparação do Ambiente: Linux

⌕ Código

Crie uma pasta lab06b e, dentro dela, o ambiente virtual com o pytest:

Terminal

```
mkdir lab06b && cd lab06b  
python3 -m venv .venv  
source .venv/bin/activate  
pip install pytest
```

Com o ambiente ativo, rode todos os testes da pasta com `pytest -q`.

Preparação do Ambiente: Windows

⌘ Código

Os mesmos passos, no PowerShell:

PowerShell

```
mkdir lab06b; cd lab06b  
python -m venv .venv  
.venv\Scripts\Activate.ps1  
pip install pytest
```

- No Prompt de Comando (cmd), a ativação é `.venv\Scripts\activate.bat`
- Se o PowerShell bloquear a ativação, rode antes `Set-ExecutionPolicy -Scope CurrentUser RemoteSigned`
- Com o ambiente ativo, rode os testes com `pytest -q`, como no Linux

Estrutura de Pastas

📁 Código

Sugestão: uma subpasta por cenário, com código e testes lado a lado. Na pasta lab06b, `pytest -q` roda os testes de todas as subpastas.

lab06b/

```
|-- tarifa/
|   |-- estacionamento.py           # Ex. 1
|   |-- test_estacionamento.py      # Ex. 1
|   |-- estacionamento_estagiario.py # Ex. 2
|   \-- test_estagiario.py          # Ex. 2
|-- isencao/
|   |-- isencao.py                  # Ex. 3
|   |-- test_isencao.py             # Ex. 3
|   |-- isento_errado.py            # Ex. 4
|   \-- test_colega.py              # Ex. 4
\-- respostas.md                    # Ex. 1 a 4
```

Contexto

Como entregar

Tabelas e respostas escritas vão em `respostas.md`, em Markdown, com uma seção por exercício. Ao final, compacte a pasta `lab06b` em um único `.zip`, sem a pasta `.venv`, e envie pelo link desta atividade no **Moodle**.

respostas.md (formato de tabela)

Exercício 3 (a)

Condicao	R1	R2
-----	----	----
Idade >= 18?	S	N
Aprovar?	S	N

Retomada das Técnicas



Classes e Valores-Limite

💡 Conceito

Partição em classes de equivalência

Técnica caixa-preta: divide o domínio de entrada em classes cujos elementos levam o sistema ao mesmo comportamento. Um representante por classe, válida ou inválida.

Análise de valor-limite

Técnica caixa-preta: testa os valores na fronteira entre duas classes e logo ao lado dela, onde se concentram erros como $<$ no lugar de $<=$.

Tabela de Decisão e Don't Care

💡 Conceito

Tabela de decisão

Técnica caixa-preta para comportamento que depende da combinação de condições: cada regra é uma combinação de valores das condições, associada a uma ação. Com n condições binárias, a tabela completa tem 2^n regras.

Don't care

Quando uma condição já decide o resultado sozinha, as demais não importam naquela regra e são marcadas com X. Regras que só diferem nessas posições se fundem em uma.

Válidos e Inválidos em Testes Separados

⌘ Código

test_notas.py

```
@pytest.mark.parametrize(
    "nota", [0.0, 0.1, 9.9, 10.0],
    ids=["min", "acima_min", "abaixo_max", "max"])
def test_nota_valida(nota):
    assert validar_nota(nota) is True

@pytest.mark.parametrize(
    "nota", [-0.1, 10.1], ids=["abaixo_min", "acima_max"])
def test_nota_invalida(nota):
    with pytest.raises(ValueError):
        validar_nota(nota)
```

Notas da aula 6, sem if no teste: um comportamento por função.¹

¹ *pytest: framework de testes para Python. `parametrize`: marcador do pytest que repete o teste para cada linha de dados.*

Estacionamento do Shopping



Contexto

O estacionamento de um shopping cobra pelo tempo de permanência, em minutos inteiros:

- 0 a 15 minutos: grátis (tolerância)
- 16 a 180 minutos: R\$ 12,00
- 181 a 720 minutos: R\$ 12,00 mais R\$ 3,00 por hora adicional ou fração
- Acima de 720 minutos: diária de R\$ 60,00
- Tempo negativo: `ValueError`
- Tempo que não seja inteiro: `TypeError`

Hora adicional ou fração: 181 minutos já pagam uma hora adicional inteira.



Exercício 1

Usando a tabela de cobrança do slide anterior:

- Implemente `calcular_tarifa(minutos)` em `estacionamento.py`.
- Liste as classes de equivalência, válidas e inválidas. A faixa de 181 a 720 minutos é uma classe só? Justifique observando o valor cobrado ao longo dela.
- Escreva a suíte em `test_estacionamento.py`: um teste parametrizado para as entradas válidas, com representantes e valores-limite de todas as fronteiras e ids descritivos, e testes separados para cada tipo de entrada inválida.

⌕ Código

estacionamento_estagiario.py

```
def calcular_tarifa(minutos):  
    if not isinstance(minutos, int):  
        raise TypeError("minutos deve ser inteiro")  
    if minutos <= 0:  
        raise ValueError("minutos nao pode ser negativo")  
    if minutos < 15:  
        return 0.00  
    if minutos <= 180:  
        return 12.00  
    if minutos < 720:  
        return 12.00 + 3.00 * ((minutos - 180) // 60)  
    return 60.00
```

Exercício 2

Um estagiário implementou a tabela de cobrança como no slide anterior, e o código já está em produção.

- Sem executar o código, encontre todas as divergências em relação à especificação. Para cada uma, dê o menor valor de entrada que a revela, o valor devolvido, o valor esperado e a correção.
- Copie a suíte do Exercício 1 para `test_estagiario.py`, importando desta versão, e rode. Ela detecta todas as divergências?
- Uma suíte com um único representante por classe válida (8, 100, 300 e 1000 minutos) detectaria alguma delas? Explique o resultado.

Isenção da Tarifa

Contexto

O shopping passa a isentar alguns clientes da tarifa:

- Veículo com credencial de vaga especial (PCD ou idoso): sempre isento
- Sem credencial: isento se a permanência for de até 240 minutos **e** o cliente tiver compras de ao menos R\$ 150,00 no dia **ou** cadastro no aplicativo do shopping
- Nos demais casos, paga o valor de `calcular_tarifa`

São quatro condições: credencial, compras $\geq$ R\$ 150,00, cadastro no aplicativo e permanência $\leq$ 240 minutos.¹

¹PCD: pessoa com deficiência.

Exercício 3

Usando a regra de isenção do slide anterior:

- Monte a tabela de decisão completa e reduza-a por don't care, justificando cada X.
- Implemente em `isencao.py` a função `isento(pcd, valor_compra, cadastro_app, minutos)`, que devolve `True` ou `False`.
- Escreva um teste parametrizado com uma linha por regra da tabela reduzida e ids "R1", "R2", e assim por diante. Sempre que compras ou permanência decidirem a regra, use valores-limite (R\$ 150,00 e R\$ 149,99; 240 e 241 minutos).

O Teste do Colega

⌕ Código

test_colega.py

```
@pytest.mark.parametrize("pcd, compra, app, minutos, esperado", [
    (True, 0.00, False, 100, True), # R1
    (False, 200.00, False, 300, False), # R2
    (False, 200.00, False, 100, True), # R3
    (False, 50.00, True, 100, True), # R4
    (False, 50.00, False, 100, False), # R5
], ids=["R1", "R2", "R3", "R4", "R5"])
def test_isento(pcd, compra, app, minutos, esperado):
    assert isento(pcd, compra, app, minutos) is esperado
```

Um colega montou a mesma tabela reduzida do Exercício 3 e escreveu este teste para ela.

O Teste do Colega



Exercício 4

O teste do colega tem uma linha por regra da tabela reduzida e passa na implementação correta.

- Escreva em `isento_errado.py` uma implementação errada de `isento`, que viole a especificação em ao menos uma das 16 regras da tabela completa, mas passe nos cinco casos do colega. Indique quais regras da tabela completa ela erra.
- Explique que suposição sobre os X da tabela reduzida o teste do colega deixou de verificar, e corrija os valores dos casos, mantendo cinco casos, para que a sua implementação errada seja detectada.
- Enuncie uma regra geral para escolher o valor concreto de uma condição marcada com X.

✓ Síntese

- As classes de equivalência dizem o que testar; os valores-limite dizem onde os defeitos aparecem
- Uma regra com arredondamento cria fronteiras dentro da própria classe, e elas também precisam de teste
- Um representante por classe não revela erros de comparação: a versão do estagiário passava em todos
- A tabela de decisão combina condições, e o don't care reduz as regras sem perder nenhum comportamento
- Cada X ainda recebe um valor concreto no teste: escolhido contra o resultado esperado, ele verifica que a condição realmente não importa

Simulação e Teste de Software

Laboratório 6b: Partição de Equivalência, Valor-Limite e Tabela de
Decisão

Prof. Luciano Rossi

Departamento de Ciência da Computação
Centro Universitário FEI

2º Semestre de 2026